MONK INTERVIEW · JOE'S ROUND · SYSTEM DESIGN

Building the *Bigger Picture*

What happens when your invoice payment page from Round 1 becomes the foundation for a full AR platform design conversation with Joe.

01 **The Zoom-Out Frame**
How Round 1 connects to everything

02 **Multi-Tenancy & Signed Tokens**
INV-001 for 10,000 companies

03 **State Sync & Race Conditions**
Payment page vs Collections Agent

04 **Replacing the Payment Stub**
Real Stripe + idempotent webhooks

05 **Concurrency & Double Payment**
Two customers, one invoice

06 **Partial Payments & Re-trigger**
Connecting partial pay back to agent

07 **Evolved Full Schema**
All components in one data model

08 **Pair Programming Extensions**
The four most likely live coding prompts

09 **Questions & Cheat Sheet**
Joe's hard questions + your smart ones

10 **The Winning Statement**
The one paragraph that ties it all together

CONTEXT

Your Round 1 prompt was to build an invoice payment page — partial payments, balance updates, stubbed processing. Joe's round builds on top of that. Every question in this guide assumes your payment page is the starting point and asks: "now how does this work at scale, with real integrations, and connected to the rest of the product?"

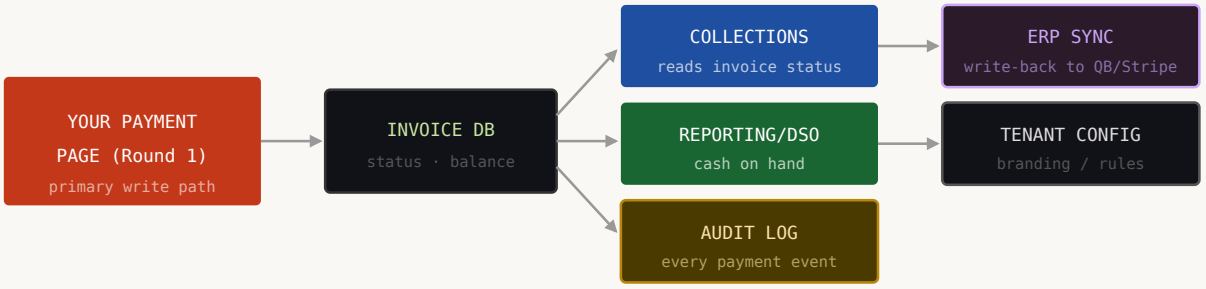
monk.com
Joe's Round
10 pages

01 The Zoom-Out Frame

MENTAL MODEL

Joe's opening will be: "*Let's take what you built last time and talk about how it fits into the bigger system.*" Your payment page is not a standalone feature — it is the **primary write path into Monk's entire AR platform**. Every design decision you made in Round 1 has downstream consequences.

YOUR PAYMENT PAGE AS ONE NODE IN A SYSTEM



THE THREE ATOMIC THINGS EVERY PAYMENT MUST DO

State this clearly and early — it frames every question that follows:

```
// ALL THREE steps must succeed together
// if any one throws, all three roll back
async function recordPayment(invoiceId: string, amountCents: number) {
  await db.transaction(async (trx) => {

    // 1. Insert payment record
    // ON CONFLICT prevents double-recording if this runs twice
    await trx.query(
      `INSERT INTO payments (id, invoice_id, amount_cents, idempotency_key)
       VALUES ($1, $2, $3, $4)
       ON CONFLICT (idempotency_key) DO NOTHING`,
      [uuid(), invoiceId, amountCents, idempotencyKey]
    );

    // 2. Update the running paid total on the invoice
    await trx.query(
      `UPDATE invoices
       SET amount_paid_cents = amount_paid_cents + $1
       WHERE id = $2`,
      [amountCents, invoiceId]
    );

    // 3. Transition status
    // This is what the collections agent reads before every outreach
    const newStatus = balanceDue === 0 ? 'paid' : 'partial';
    await trx.query(
      `UPDATE invoices SET status = $1 WHERE id = $2`,
      [newStatus, invoiceId]
    );
  });
}
```

WHY THIS MATTERS TO JOE

If the balance updates but the status does not, the AI collections agent will send a dunning email to a customer who already paid. That single bug destroys trust and is the #1 failure mode of any AR product. Atomicity here is non-negotiable.

TECHNICAL

System Thinking

Can you trace a single payment write through all downstream systems?

BUSINESS

Impact Awareness

Do you know why correctness matters — not just that it does?

COLLABORATION

Build on Prior Work

Reference your Round 1 choices explicitly. Show continuity.

02 Multi-Tenancy & Signed Payment Tokens

SCALING QUESTION

Joe's prompt: *"Your page works for one invoice. How does it work for 10,000 companies, each with their own branding, invoice IDs, and customers?"*

TWO PROBLEMS WITH YOUR ROUND 1 URL

PROBLEM 1 — ID COLLISIONS

INV-001 is not globally unique

Acme Corp has INV-001. So does every other customer. The ID is unique within a tenant, not globally. Your URL needs tenant context baked in.

PROBLEM 2 — AUTH VS PUBLIC

Customers have no Monk account

Payment links are opened by the end customer who received the invoice. They have no login. You need public access that is still secure and scoped to exactly one invoice.

SOLUTION: SIGNED PAYMENT TOKEN

`Invoice Created` → `Generate JWT` → `Store token hash` → `Email link to customer` → `GET /pay/{token}` → `Decode + verify` → `Render invoice`

```
// Token payload — JWT signed with HS256
{
  "invoice_id": "uuid",
  "tenant_id":  "uuid",
  // 30-day expiry
  "exp":        1780000000,
  // for revocation — stored hashed in DB
  "jti":        "unique-token-id"
}

// Route — no session auth needed; token IS the credential
GET /pay/:token
  // tamper-proof — reject if signature invalid
  verify HMAC signature
  // link not expired
  check exp not passed
  // not revoked (voided invoices)
  check jti not in void set
  fetch invoice WHERE id = payload.invoice_id AND tenant_id = payload.tenant_id
  render page with tenant branding from tenant_configs table
```

SECURITY PROPERTIES OF THIS DESIGN

PROPERTY	HOW ACHIEVED	WHAT IT PREVENTS
Tamper-proof	HMAC-SHA256 signature	Attacker modifying invoice_id or amount in URL
Expirable	`exp` claim + server-side check	Old payment links staying live indefinitely
Revokable	`jti` stored in DB, checked on every request	Voided invoices still accepting payments
Tenant-scoped	`tenant_id` in payload, verified on DB fetch	Cross-tenant data leakage via guessed IDs
Opaque	Base64 JWT — no raw IDs visible in URL	Customer enumerating other invoice IDs

PER-TENANT CUSTOMIZATION

Loaded from tenant_configs

Logo URL, brand color, company name, payment instructions language, currency format. All resolved from `tenant_id` at render time. Zero code deploys for customer customization.

TOKEN EXPIRY UX

Resend Flow Needed

Expired token = dead link. You need a "resend payment link" endpoint that generates a new JWT and re-emails it. The old token should be revoked via its jti on resend.

SAY THIS TO JOE

"I'd move away from sequential INV-001 IDs for the URL immediately — they leak business intelligence. Opaque signed tokens give you security, tenant isolation, expiry, and revocability in one pattern. The token is also self-contained enough that you don't need a session for the customer at all."

KNOW COLD

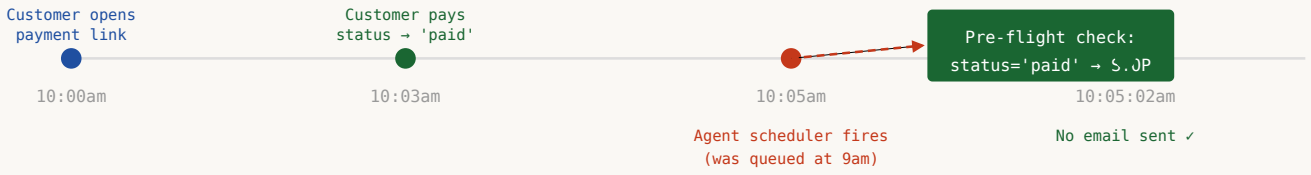
- JWT structure (header.payload.sig)
- HS256 vs RS256 tradeoffs
- Token revocation via jti + DB set
- Row-level security by tenant_id
- Opaque token vs self-contained JWT
- Link expiry + resend UX pattern

03 State Sync & The Race Condition

MOST CRITICAL DESIGN

Joe's prompt: *"When a customer pays on your portal, how does the AI collections agent know to stop following up?"*

THE RACE CONDITION VISUALIZED



INVOICE STATUS STATE MACHINE

DRAFT → SENT → OVERDUE → FOLLOW-UP 1 → FOLLOW-UP 2 → PARTIAL → PAID or → VOID

PRE-FLIGHT CHECK PATTERN — THE FIX

```
// Agent job: always re-read status right before dispatch
async function dispatchFollowUp(invoiceId) {

  // FOR UPDATE locks the row
  // prevents a payment from racing between the read and the send
  const invoice = await db.query(
    'SELECT status, balance_due_cents FROM invoices WHERE id=$1 FOR UPDATE',
    [invoiceId]
  );

  // Pre-flight gates — bail if state changed since job was queued
  // paid since job was created
  if (invoice.status === 'paid') return;
  // voided invoice
  if (invoice.status === 'void') return;
  // safety net
  if (invoice.balance_due_cents === 0) return;

  // Only now is it safe to generate and dispatch the follow-up
  await sendFollowUp(invoice);
  await logOutreachEvent(invoiceId);
}
```

CASCADING STATE CHANGES FROM YOUR PAYMENT PAGE

PAYMENT EVENT	NEW INVOICE STATUS	AGENT RESPONSE
Full payment (balance = 0)	paid	Cancel all queued follow-ups immediately
Partial payment (balance > 0)	partial	Reset follow-up clock, shift to "appreciative" tone
Overpayment received	paid + credit	Cancel follow-up; flag credit for human review

PAYMENT EVENT	NEW INVOICE STATUS	AGENT RESPONSE
Chargeback / payment reversed	overdue	Re-trigger entire follow-up sequence from start

KNOW COLD

* SELECT FOR UPDATE (row-level lock)
* TOCTOU race condition
* Atomic multi-table transactions
* Pre-flight check pattern
* State machine valid transitions
* Job cancellation in BullMQ

04 Replacing the Stub with Real Stripe

INTEGRATION

Joe's prompt: *"You stubbed out payment processing in Round 1. Walk me through how you'd replace it with Stripe."*

PAYMENT INTENT FLOW

Client loads page → POST /create-intent → Stripe API → client_secret back → Stripe Elements → Customer enters card → Stripe webhook fires

```
// Step 1: Your server creates a payment intent
// POST /api/pay/:token
const intent = await stripe.paymentIntents.create({
  amount:   requestedAmountCents,
  currency: 'usd',
  metadata: { invoice_id: invoice.id, tenant_id: invoice.tenant_id }
});
// send client_secret to browser so Stripe Elements can complete the payment
return { clientSecret: intent.client_secret };


// Step 2: Stripe calls YOUR webhook when payment completes
// POST /webhooks/stripe

// FIRST: verify signature — reject anything unsigned
// body must be raw buffer, not parsed JSON — or signature check fails
const event = stripe.webhooks.constructEvent(
  req.body,
  req.headers['stripe-signature'],
  process.env.STRIPE_WEBHOOK_SECRET
);

// respond 200 immediately — process async after
// Stripe retries if your handler takes longer than 30s
res.json({ received: true });

if (event.type === 'payment_intent.succeeded') {
  await recordPaymentIdempotent({
    // stripe intent ID is your idempotency key
    stripeIntentId: event.data.object.id,
    invoiceId:      event.data.object.metadata.invoice_id,
    amountCents:    event.data.object.amount_received
  });
}
```

IDEMPOTENT WEBHOOK HANDLER

Stripe guarantees **at-least-once delivery** — the same webhook can arrive twice on retries. Your handler must handle duplicates silently:

```
// ON CONFLICT silently ignores duplicate events
// If rowsAffected === 0: duplicate — skip all downstream updates
INSERT INTO payments (stripe_intent_id, invoice_id, amount_cents)
VALUES ($1, $2, $3)
ON CONFLICT (stripe_intent_id) DO NOTHING;
```

THE UI CONFIRMATION PROBLEM

After the customer clicks Pay, there is a 1–3 second delay before your webhook fires and updates the DB. Your UI cannot just reload immediately. Two options:

OPTION A — POLLING (SIMPLE)

GET /invoice/status every 2s

After Stripe confirms client-side, poll your API for up to 30s. When status flips to `paid`, show the success state. Works everywhere, slight delay is acceptable in B2B contexts.

OPTION B — SERVER-SENT EVENTS

SSE /invoice/{id}/stream

Server pushes an update the moment the webhook is processed. Instant, no polling. More complex. Better UX if the payment page has high traffic or latency-sensitive customers.

WEBHOOK SECURITY CHECKLIST

CHECK	HOW	WITHOUT IT
Verify signature	`stripe.webhooks.constructEvent()`	Anyone can POST fake payment events to your endpoint
Raw body	Parse as Buffer before JSON.parse	Signature check always fails
Idempotency key	Unique constraint on `stripe_intent_id`	Double payment record on Stripe retry
Respond 200 fast	Return before processing async	Stripe retries if your handler is slow (>30s)

KNOW COLD

* Payment Intent vs Charge API
* Webhook HMAC signature
* At-least-once delivery guarantee
* Idempotent DB upsert pattern
* SSE vs WebSocket vs polling
* PCI: never touch raw card data

05 Concurrency & Double Payment Prevention

HARD EDGE CASE

Joe's prompt: *"Two customers open the same invoice link at the exact same time and both try to pay in full. What happens?"*

Without protection, both transactions read `balance_due = 50000`, both succeed, and you have two full payments against one invoice — a corrupt AR record.

THREE SOLUTIONS, RANKED BY SIMPLICITY

SOLUTION 1 — RECOMMENDED

SELECT FOR UPDATE

Lock the invoice row when the transaction begins. Second transaction blocks until first commits, then reads updated (paid) state and aborts.

SOLUTION 2

Optimistic Locking

Add `version INT`. Update includes `WHERE version = $current`. Second write gets 0 rows updated and retries with fresh state.

SOLUTION 3

Idempotency Key

Client generates UUID on page load. Unique constraint prevents duplicate records from the same session but does not stop two genuinely concurrent sessions.

SELECT FOR UPDATE — RECOMMENDED IMPLEMENTATION

```
async function recordPaymentSafe(invoiceId, amountCents) {
  return db.transaction(async (trx) => {

    // 1. Lock the row
    // Second concurrent transaction WAITS HERE until this one commits
    const { rows: [inv] } = await trx.query(
      'SELECT * FROM invoices WHERE id = $1 FOR UPDATE',
      [invoiceId]
    );

    // 2. Pre-flight — validate state with the now-locked, fresh row
    if (inv.status === 'paid') throw new Error('Already paid');
    if (inv.status === 'void') throw new Error('Invoice voided');
    if (amountCents > inv.balance_due_cents) throw new Error('Exceeds balance');
    if (amountCents <= 0) throw new Error('Amount must be positive');

    // 3. Write atomically — now safe to proceed
    await trx.query(
      'INSERT INTO payments (invoice_id, amount_cents) VALUES ($1, $2)',
      [invoiceId, amountCents]
    );

    const newStatus = amountCents >= inv.balance_due_cents ? 'paid' : 'partial';
    await trx.query(
      'UPDATE invoices SET amount_paid_cents = amount_paid_cents + $1, status = $2 WHERE id = $3',
      [amountCents, newStatus, invoiceId]
    );

    // When the second concurrent transaction acquires the lock here,
    // it reads status='paid' and throws — no double payment possible
  });
}
```

WHICH TO RECOMMEND TO JOE AND WHY

APPROACH	BEST FOR	KEY DRAWBACK
SELECT FOR UPDATE	Low-to-medium concurrency; most B2B invoicing scenarios	Holds DB lock; bad under extreme write contention
Optimistic locking	High read:write ratio; many readers, few writers	Requires retry loop; degrades under high write concurrency
Idempotency key only	Same-client retries (browser refresh, network failure)	Doesn't protect against two genuinely different clients

SAY THIS TO JOE

"Two people simultaneously paying the same B2B invoice is vanishingly rare in practice. The real problem is browser double-submissions and network retries from a single session — idempotency keys solve that. I'd use SELECT FOR UPDATE for the theoretical concurrent case because it's simple, correct, and adds no infrastructure. I'd revisit if this becomes a bottleneck, which it almost certainly won't."

06 Partial Payments & Re-triggering the Agent

CORE PRODUCT LOGIC

Joe's prompt: *"You allowed partial payments in Round 1. After a partial payment, the agent needs to keep following up — but differently. How does that work?"*

A partial payment is a *positive signal* — the customer wants to pay, just not in full yet. The agent must acknowledge this, de-escalate, reset the clock, and reference the remaining balance — not resend a generic dunning email.

PARTIAL PAYMENT STATE TRANSITION

OVERDUE → FOLLOW-UP 1 (generic dunning) → PARTIAL PAYMENT ($X received) → STATUS: PARTIAL → PARTIAL F/U 1 (acknowledge + remind) → PAID

WHAT CHANGES AFTER A PARTIAL PAYMENT

DIMENSION	BEFORE (OVERDUE)	AFTER PARTIAL PAY (PARTIAL)
Message tone	"INV-001 is overdue for $5,000"	"Thanks for your $2,000 payment. Remaining balance: $3,000"
Follow-up clock	Counting since due date	**Resets to 0** from partial payment date
Urgency	Escalating with each follow-up	De-escalated — customer shown good faith
Amount referenced	Full original amount	Remaining `balance_due_cents` only
Payment link	Pre-fills full invoice amount	Pre-fills remaining balance

TYPESCRIPT TYPES FOR PARTIAL PAYMENT TRACKING

```typescript
// Extends your Round 1 Invoice type with partial payment tracking

type InvoiceStatus =
  | 'draft'
  | 'sent'
  | 'overdue'
  | 'partial'
  | 'paid'
  | 'void';

interface Invoice {
  id: string;
  tenantId: string;
  status: InvoiceStatus;
  originalAmountCents: number;
  amountPaidCents: number;
  // always derived — never store independently or it will drift
  get balanceDue(): number;
  dueDate: Date;
  // set when first partial payment is recorded
  partialPaidAt: Date | null;
  // agent resets follow-up clock to this date on each partial payment
  followupResetAt: Date | null;
  // scheduler cursor — null means no follow-up is pending
  nextFollowupAt: Date | null;
  followupCount: number;
}
```

```
// One row per invoice a payment touches
// supports both partial pays and one payment covering multiple invoices
interface PaymentAllocation {
  id: string;
  paymentId: string;
  invoiceId: string;
  amountCents: number;
  allocatedAt: Date;
}
```

AGENT CLOCK RESET LOGIC

```
// Agent computes days overdue from whichever event is most recent
// if no partial payment yet, count from due date
const clockStartDate = invoice.partial_paid_at ?? invoice.due_date;

// Context injected into LLM prompt for tone personalisation
{
  has_partial_payment: invoice.partial_paid_at !== null,
  amount_paid_cents:   invoice.amount_paid_cents,
  amount_remaining:    invoice.balance_due_cents,
  tone: invoice.partial_paid_at ? "appreciative_reminder" : "standard_dunning"
}
```

EDGE CASE 1

Multiple Partial Payments

Customer pays $1k, $1k, $1k against a $5k invoice. Clock should reset on the *most recent* partial payment. Use `MAX(allocated_at)` from payment_allocations — update `followup_reset_at` on every partial payment.

EDGE CASE 2

Stale Amount on Page

Customer opens link showing $3,000 remaining. While they're reading it, another partial payment is made. Submit-time validation must check current `balance_due_cents`, not the page-load snapshot.

SAY THIS TO JOE

"A partial payment is one of the best signals you can get from a customer — it shows intent to pay. Sending the same dunning email after receiving a partial payment is a relationship mistake. The system should acknowledge the payment, de-escalate, and make it easy to pay the remainder — not treat the customer like they haven't responded at all."

07 The Evolved Full Schema

REFERENCE

How your Round 1 types evolve once you add multi-tenancy, signed tokens, concurrency protection, partial payments, and audit logging. Every field is annotated with why it exists.

```
// ── TENANT ──────────────────────────────────────────

interface Tenant {
  id: string;
  name: string;
  logoUrl: string;
  // hex color used to theme the customer-facing payment page
  brandColor: string;
  // Stripe Connect account — needed for multi-tenant payouts
  stripeAcctId: string;
}


// ── INVOICE ─────────────────────────────────────────

type InvoiceStatus =
  | 'draft'
  | 'sent'
  | 'overdue'
  | 'partial'
  | 'paid'
  | 'void';

interface LineItem {
  id: string;
  description: string;
  quantity: number;
  unitPriceCents: number;
}

interface Invoice {
  id: string;
  tenantId: string;
  customerId: string;
  // display only — "INV-001" is NOT globally unique across tenants
  displayNumber: string;
  status: InvoiceStatus;
  lineItems: LineItem[];
  originalAmountCents: number;
  amountPaidCents: number;
  // derived getter — never store this directly or it will drift
  get balanceDue(): number;
  dueDate: Date;
  issuedAt: Date;
  paidAt: Date | null;
  // set when the first partial payment is recorded
  partialPaidAt: Date | null;
  // agent resets its follow-up clock to this on every partial payment
  followupResetAt: Date | null;
  // scheduler cursor — null means no follow-up currently pending
  nextFollowupAt: Date | null;
```

```
  followupCount: number;
  // incremented on every write — used for optimistic locking
  version: number;
  // hashed JWT jti — so we can revoke a token without decoding it
  paymentToken: string | null;
  tokenExpiresAt: Date | null;
}


// ── PAYMENT ──────────────────────────────────────────

type PaymentStatus =
  | 'pending'
  | 'succeeded'
  | 'failed'
  | 'refunded';

interface Payment {
  id: string;
  tenantId: string;
  // used as idempotency key — ON CONFLICT on this field in the DB
  stripeIntentId: string | null;
  amountCents: number;
  status: PaymentStatus;
  paidAt: Date | null;
  payerIp: string;
}


// ── PAYMENT ALLOCATION ───────────────────────────────
// maps one Payment to one or more Invoices
// one payment can cover multiple invoices, or partially cover one

interface PaymentAllocation {
  id: string;
  paymentId: string;
  invoiceId: string;
  amountCents: number;
  allocatedAt: Date;
}


// ── AUDIT LOG ────────────────────────────────────────
// append-only — rows are never updated or deleted

type ActorType =
  | 'customer'
  | 'system'
  | 'agent'
  | 'user';

type AuditAction =
  | 'invoice.paid'
  | 'invoice.partial'
  | 'invoice.voided'
  | 'payment.recorded'
  | 'outreach.sent'
  | 'config.updated';
```

```typescript
interface AuditLog {
  id: string;
  tenantId: string;
  actorType: ActorType;
  actorId: string | null;
  action: AuditAction;
  // 'invoice' | 'payment' | 'config'
  resourceType: string;
  resourceId: string;
  // before/after snapshot of changed fields
  metadata: Record<string, unknown>;
  createdAt: Date;
}
```

KEY DESIGN POINT

`balanceDue` *is a* ***derived getter*** *— always computed as* `originalAmountCents - amountPaidCents`*. Never store it independently or it will drift. In your DB layer, implement this as a computed/virtual field for the same guarantee. Eliminates an entire class of consistency bugs.*

08 Pair Programming Extensions

WHAT YOU WILL CODE LIVE

Joe's round ends with live coding that directly extends the system design you just discussed. The pair programming prompt will be one of these four — ranked by likelihood:

EXTENSION 1 (MOST LIKELY) — SIGNED PAYMENT TOKEN

```
import jwt from 'jsonwebtoken';
import { createHash } from 'crypto';

// Called when invoice is sent to customer
async function generatePaymentToken(invoice) {
  const token = jwt.sign(
    { invoice_id: invoice.id, tenant_id: invoice.tenant_id },
    process.env.JWT_SECRET,
    { expiresIn: '30d' }
  );

  // Store hashed — so raw token is not exposed if DB is ever read
  const tokenHash = createHash('sha256').update(token).digest('hex');
  await db.query(
    'UPDATE invoices SET payment_token=$1 WHERE id=$2',
    [tokenHash, invoice.id]
  );

  // Return raw token — this is what goes in the email link
  return token;
}

// Called on GET /pay/:token
async function verifyPaymentToken(token) {
  // throws if expired or tampered
  const payload = jwt.verify(token, process.env.JWT_SECRET);

  const tokenHash = createHash('sha256').update(token).digest('hex');
  const { rows: [inv] } = await db.query(
    'SELECT * FROM invoices WHERE id=$1 AND payment_token=$2 AND status != $3',
    [payload.invoice_id, tokenHash, 'void']
  );

  if (!inv) throw new Error('Invalid, expired, or revoked token');
  return inv;
}
```

EXTENSION 2 — IDEMPOTENT STRIPE WEBHOOK

```
app.post('/webhooks/stripe', express.raw({ type: 'application/json' }), async (req, res) => {
  try {
    const event = stripe.webhooks.constructEvent(
      req.body,
      req.headers['stripe-signature'],
      process.env.STRIPE_SECRET
    );
```

```
    // respond fast — Stripe retries if you take longer than 30s
    res.json({ received: true });

    if (event.type === 'payment_intent.succeeded') {
      const pi = event.data.object;
      await db.transaction(async (trx) => {
        const r = await trx.query(
          'INSERT INTO payments(stripe_intent_id,amount_cents) VALUES($1,$2) ON CONFLICT DO NOTHING',
          [pi.id, pi.amount_received]
        );

        // duplicate event — skip all downstream work
        if (r.rowCount === 0) return;

        await recordPaymentSafe(trx, pi.metadata.invoice_id, pi.amount_received);
      });
    }
  } catch (e) {
    res.status(400).send('Bad signature');
  }
});
```

EXTENSION 3 — INVOICE STATE MACHINE

```
const VALID_TRANSITIONS = {
  draft:   ['sent', 'void'],
  sent:    ['overdue', 'paid', 'void'],
  overdue: ['partial', 'paid', 'void'],
  partial: ['paid', 'void'],
  // terminal states — no exit allowed
  paid:    [],
  void:    [],
};

function transition(invoice, newStatus) {
  const allowed = VALID_TRANSITIONS[invoice.status] ?? [];
  if (!allowed.includes(newStatus)) {
    throw new Error(`Invalid transition: ${invoice.status} -> ${newStatus}`);
  }
  return { ...invoice, status: newStatus };
}

// Usage
// const updated = transition(invoice, 'paid'); — throws if transition is not allowed
```

09–10 Questions, Cheat Sheet & The Winning Statement

FINAL PREP

JOE'S HARDEST QUESTIONS

- Payment records but status update fails — what's the system state?
- Customer reopens a paid invoice link — what should they see?
- How do you prevent a voided invoice from accepting payment?
- What if your Stripe webhook never arrives at all?
- Payment is refunded — what status does the invoice revert to?
- How would you let a tenant customize the payment page without a deploy?
- The payment token expires while customer is filling in the form — then what?
- How do you audit "who voided this invoice and when"?
- Two partial payments arrive simultaneously from the same customer — idempotent?
- How do you add ACH support alongside card payments?

YOUR SMART QUESTIONS FOR JOE

- When extending Round 1 today — API + DB, or do you want UI too?
- Has Monk had any double-payment incidents in production?
- Does the token expiry need to be configurable per tenant?
- What's the most common reason a payment link fails today?
- Is the payment page hosted on Monk's domain or embedded in the customer's app?
- How does Monk handle disputes — in-system flag or bank chargeback flow?
- Is there a requirement to support currencies other than USD?
- What does "refund" look like — does Monk initiate it, or just record it?

RAPID-FIRE CONCEPT CHEAT SHEET

CONCEPT	ONE-LINE DEFINITION	WHERE IT APPLIES
Idempotency key	Unique key making repeat operations safe; second call is a no-op	Stripe webhook, payment recording
SELECT FOR UPDATE	Row-level exclusive lock held for the entire transaction	Concurrent payment prevention
Optimistic locking	Version field prevents stale writes; 0 rows = someone else changed it	Alternative to pessimistic lock
Atomic transaction	All-or-nothing write; all steps succeed or all roll back	Payment + balance + status update
Pre-flight check	Re-read state immediately before acting — never trust cached state	Agent before every outreach
Generated column	Value always computed from other columns; structurally impossible to drift	`balance_due = original - paid`
TOCTOU race	State changes between when you checked it and when you act on it	Agent checking status then sending
At-least-once delivery	Message guaranteed to arrive; may arrive more than once	Stripe webhooks — must be idempotent
State machine	Explicit set of states + only allowed transitions between them	Invoice status lifecycle
Signed token	Opaque credential with embedded claims and cryptographic tamper protection	Payment page URL authentication

THE WINNING STATEMENT — SAY THIS OUT LOUD

*"The payment page isn't just a UI — it's the **primary write path into the entire AR system**. Every payment must atomically do three things in one transaction: insert the payment record, update the invoice balance, and transition the invoice status. If those three are ever out of sync, every downstream system — the AI collections agent, DSO reporting, reconciliation — is working off corrupt data. I'd protect that atomicity above everything else in this codebase, and I'd make the status field the single source of truth that every other system reads."*

MASTER THESE BEFORE THE INTERVIEW

- JWT sign + verify in Node.js
- Postgres transactions (BEGIN/COMMIT)
- SELECT FOR UPDATE syntax
- ON CONFLICT DO NOTHING / UPDATE
- Stripe webhook signature verify
- State machine pattern in TypeScript
- Generated columns in Postgres
- Row-level tenant_id scoping
- SSE / EventSource in browser